

C++ String & Number Manipulation Programs

Compiled Reference Manual & Code Execution Guides

1. Palindrome Number Check

File: tut78.cpp

Checks whether a given number reads the same backward as forward by reversing the digits mathematically.

```
#include <iostream>
using namespace std;

int main() {
    int num, original, reverse = 0, remainder;
    cout << "enter a number --->";
    cin >> num;

    original = num;
    while (num != 0) {
        remainder = num % 10;
        reverse = reverse * 10 + remainder;
        num = num / 10;
    }

    if (original == reverse)
        cout << "it is a palindrome number";
    else
        cout << "it is not a palindrome number";

    return 0;
}
```

TERMINAL OUTPUT

```
enter a number --->121
it is a palindrome number
```

2. Anagram Check (Using Sorting)

File: tut79.cpp

Determines if two strings are anagrams of each other by checking lengths and sorting both strings alphabetically.

```
#include <iostream>
#include <algorithm>
using namespace std;

int main() {
    string str1, str2;
    cout << "Enter first string: ";
    cin >> str1;
    cout << "Enter second string: ";
    cin >> str2;

    // Check length
    if (str1.length() != str2.length()) {
        cout << "Not Anagrams";
        return 0;
    }

    // Sort both strings
    sort(str1.begin(), str1.end());
    sort(str2.begin(), str2.end());

    // Compare
    if (str1 == str2)
        cout << "Anagrams";
    else
        cout << "Not Anagrams";

    return 0;
}
```

TERMINAL OUTPUT

```
Enter first string: listen
Enter second string: silent
Anagrams
```

3. Pangram Check

File: tut80.cpp

A pangram is a sentence containing every letter of the alphabet at least once.

```
#include <iostream>
#include <cctype>
using namespace std;

int main()
{
    string str;
    getline(cin, str);

    int seen[26] = {0};
    int count = 0;

    for (int i = 0; i < str.length(); i++)
    {
        char ch = tolower(str[i]);
        if (ch >= 'a' && ch <= 'z')
        {
            if (!seen[ch - 'a'])
            {
                seen[ch - 'a'] = 1;
                count++;
            }
        }
    }

    if (count == 26)
        cout << "Pangram";
    else
        cout << "Not Pangram";

    return 0;
}
```

TERMINAL OUTPUT

Not Pangram

4. Isogram Check (Using Hash Set)

File: tut81.cpp

Checks if a string has no repeating letters using an unordered_set.

```
#include <iostream>
#include <unordered_set>
#include <cctype>
using namespace std;

bool isIsogram(string str) {
    unordered_set<char> seen;

    for (char ch : str) {
        if (isalpha(ch)) {
            ch = tolower(ch);
            if (seen.count(ch))
                return false;
            seen.insert(ch);
        }
    }
    return true;
}

int main() {
    string str;
    getline(cin, str);

    if (isIsogram(str))
        cout << "Isogram" << endl;
    else
        cout << "Not an Isogram" << endl;

    return 0;
}
```

TERMINAL OUTPUT

Not an Isogram

5. Heterogram Check

File: tut82.cpp

A heterogram is a word, phrase, or sentence in which no letter of the alphabet occurs more than once.

```
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

int main() {
    string str;
    getline(cin, str);

    int freq[26] = {0};

    for (int i = 0; i < str.length(); i++) {
        char ch = tolower(str[i]);
        if (ch >= 'a' && ch <= 'z') {
            freq[ch - 'a']++;
            if (freq[ch - 'a'] > 1) {
                cout << "Not Heterogram";
                return 0;
            }
        }
    }

    cout << "Heterogram";
    return 0;
}
```

TERMINAL OUTPUT

Not Heterogram

6. Semordnilap Check

File: tut83.cpp

A semordnilap is a word or phrase that spells a different word when reversed (e.g., 'stressed' -> 'desserts').

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string a, b;
    cin >> a >> b;

    int n = a.length();

    if (a == b) {
        cout << "Not Semordnilap";
        return 0;
    }

    if (n != b.length()) {
        cout << "Not Semordnilap";
        return 0;
    }

    for (int i = 0; i < n; i++) {
        if (a[i] != b[n - i - 1]) {
            cout << "Not Semordnilap";
            return 0;
        }
    }

    cout << "Semordnilap";
    return 0;
}
```

TERMINAL OUTPUT

Not Semordnilap

7. Capitalize First Letter of Each Word

File: tut84.cpp

Extracts and prints the first letter of each word in uppercase.

```
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

int main() {
    string str;
    getline(cin, str);

    bool newWord = true;

    for (int i = 0; i < str.length(); i++) {
        if (newWord && isalpha(str[i])) {
            cout << (char)toupper(str[i]);
        }
        newWord = (str[i] == ' ');
    }

    return 0;
}
```

TERMINAL OUTPUT

CCIGTTI

8. Lipogram Check

File: tut85.cpp

A lipogram is a text that deliberately excludes a particular letter of the alphabet.

```
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

int main() {
    string str;
    char ch;

    getline(cin, str);
    cin >> ch;

    ch = tolower(ch);

    for (int i = 0; i < str.length(); i++) {
        if (tolower(str[i]) == ch) {
            cout << "Not Lipogram";
            return 0;
        }
    }

    cout << "Lipogram";
    return 0;
}
```

TERMINAL OUTPUT

Not Lipogram

9. Abecedarian Word Check

File: tut86.cpp

Checks if the letters of a word are arranged in alphabetical order.

```
#include <iostream>
#include <string>
#include <cctype>
using namespace std;

int main() {
    string str;
    cin >> str;

    for (int i = 1; i < str.length(); i++) {
        if (tolower(str[i]) < tolower(str[i - 1])) {
            cout << "Not Abecedarian";
            return 0;
        }
    }

    cout << "Abecedarian";
    return 0;
}
```

TERMINAL OUTPUT

Abecedarian

10. Anagram Check (Using Frequency Array)

File: tut87.cpp

An alternative approach to check anagrams using a frequency array of 256 characters for efficient $O(N)$ execution.

```
#include <iostream>
using namespace std;

int main() {
    string a, b;
    cin >> a >> b;

    int freq[256] = {0};

    for (char ch : a) {
        freq[(unsigned char)ch]++;
    }

    for (char ch : b) {
        freq[(unsigned char)ch]--;
    }

    for (int i = 0; i < 256; i++) {
        if (freq[i] != 0) {
            cout << "Not Anagram";
            return 0;
        }
    }

    cout << "Anagram";
    return 0;
}
```

TERMINAL OUTPUT

Not Anagram